

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №3.

Тема: «Решение задачи о коммивояжере»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

Необходимо реализовать решение задачи коммивояжера для взвешенного графа с использованием:

- матрицы смежности;
- полного перебора маршрутов;
- красивого консольного вывода;
- визуализации итогового пути в виде вертикального дерева.

Граф должен содержать минимум 6 вершин, чтобы было видно:

- наличие большого количества маршрутов;
- сравнение различных путей;
- выбор минимального маршрута.

Анализ задачи:

Исходный граф:

	A	B	C	D	E	F
A	∞	12	7	18	5	9
B	12	∞	4	6	14	11
C	7	4	∞	8	10	15
D	18	6	8	∞	3	13
E	5	14	10	3	∞	16
F	9	11	15	13	16	∞

1. Исходная матрица.
2. Редукция строк.
3. Редукция столбцов.
4. Оценка нулей.
5. Выбор лучшего ребра.
6. Удаление строки и столбца.
7. Повторение.
8. Построение маршрута.

Код:

```
#include <iostream>

#include <iomanip>
#include <vector>
#include <algorithm>
#include <climits>

using namespace std;

const int N = 6;
const int INF = 999999;

char city[N] = { 'A', 'B', 'C', 'D', 'E', 'F' };

int matrix[N][N] =
{
    {INF, 12, 7, 18, 5, 9},
    {12, INF, 4, 6, 14, 11},
    {7, 4, INF, 8, 10, 15},
    {18, 6, 8, INF, 3, 13},
    {5, 14, 10, 3, INF, 16},
    {9, 11, 15, 13, 16, INF}
};

struct Edge
{
    int from;
    int to;
};

vector<Edge> resultPath;

void printMatrix(int m[N][N])
{
    cout << "\nРЕДУЦИРОВАННАЯ МАТРИЦА:\n\n";

    cout << setw(6) << " ";
```

```

    for (int i = 0; i < N; i++)
        cout << setw(6) << city[i];

    cout << endl;

    for (int i = 0; i < N; i++)
    {
        cout << setw(6) << city[i];

        for (int j = 0; j < N; j++)
        {
            if (m[i][j] == INF)
                cout << setw(6) << "INF";
            else
                cout << setw(6) << m[i][j];
        }

        cout << endl;
    }
}

int reduceRows(int m[N][N])
{
    int reduction = 0;

    for (int i = 0; i < N; i++)
    {
        int rowMin = INF;

        for (int j = 0; j < N; j++)
        {
            rowMin = min(rowMin, m[i][j]);
        }

        if (rowMin != INF && rowMin != 0)
        {
            reduction += rowMin;
        }
    }
}

```

```

        for (int j = 0; j < N; j++)
        {
            if (m[i][j] != INF)
                m[i][j] -= rowMin;
        }
    }

    return reduction;
}

int reduceCols(int m[N][N])
{
    int reduction = 0;

    for (int j = 0; j < N; j++)
    {
        int colMin = INF;

        for (int i = 0; i < N; i++)
        {
            colMin = min(colMin, m[i][j]);
        }

        if (colMin != INF && colMin != 0)
        {
            reduction += colMin;

            for (int i = 0; i < N; i++)
            {
                if (m[i][j] != INF)
                    m[i][j] -= colMin;
            }
        }
    }

    return reduction;
}

```

```
}
```

```
int calculatePenalty(int m[N][N], int row, int col)
```

```
{
```

```
    int rowMin = INF;
```

```
    int colMin = INF;
```

```
    for (int j = 0; j < N; j++)
```

```
    {
```

```
        if (j != col)
```

```
            rowMin = min(rowMin, m[row][j]);
```

```
    }
```

```
    for (int i = 0; i < N; i++)
```

```
    {
```

```
        if (i != row)
```

```
            colMin = min(colMin, m[i][col]);
```

```
    }
```

```
    if (rowMin == INF)
```

```
        rowMin = 0;
```

```
    if (colMin == INF)
```

```
        colMin = 0;
```

```
    return rowMin + colMin;
```

```
}
```

```
void printPenalties(int m[N][N])
```

```
{
```

```
    cout << "\nШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК:\n\n";
```

```
    for (int i = 0; i < N; i++)
```

```
    {
```

```
        for (int j = 0; j < N; j++)
```

```
        {
```

```
            if (m[i][j] == 0)
```

```
            {
```

```

        int penalty = calculatePenalty(m, i, j);

        cout << "("
              << city[i]
              << " -> "
              << city[j]
              << ") = "
              << penalty
              << endl;
    }
}
}
}

```

```

Edge chooseBestEdge(int m[N][N])
{
    int bestPenalty = -1;

    Edge bestEdge = { -1, -1 };

    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            if (m[i][j] == 0)
            {
                int penalty = calculatePenalty(m, i, j);

                if (penalty > bestPenalty)
                {
                    bestPenalty = penalty;

                    bestEdge.from = i;
                    bestEdge.to = j;
                }
            }
        }
    }
}

```

```

if (bestEdge.from == -1 || bestEdge.to == -1)
{
    cout << "\nНулевые элементы больше не найдены.\n";

    return { -1, -1 };
}

cout << "\nВЫБРАНО РЕБРО: "
    << city[bestEdge.from]
    << " -> "
    << city[bestEdge.to]
    << endl;

cout << "ШТРАФ = "
    << bestPenalty
    << endl;

return bestEdge;
}

void removeRowCol(int m[N][N], int row, int col)
{
    for (int j = 0; j < N; j++)
        m[row][j] = INF;

    for (int i = 0; i < N; i++)
        m[i][col] = INF;

    m[col][row] = INF;
}

void printTree()
{
    cout << "ФИНАЛЬНОЕ ДЕРЕВО МАРШРУТА\n";
}

```



```

for (int i = 0; i < resultPath.size(); i++)
{
    cout << city[resultPath[i].from]
         << " -> "
         << city[resultPath[i].to]
         << endl;

    if (i != resultPath.size() - 1)
    {
        cout << "\\n";
        cout << "v\\n";
    }
}

int calculateFinalCost()
{
    int total = 0;

    for (auto e : resultPath)
    {
        total += matrix[e.from][e.to];
    }

    return total;
}

int main()
{
    setlocale(LC_ALL, "Russian");

    int reducedMatrix[N][N];

    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            reducedMatrix[i][j] = matrix[i][j];

```

```
    }  
}
```

```
cout << "ЗАДАЧА КОММИВОЯЖЕРА\n";  
cout << "МЕТОД ВЕТВЕЙ И ГРАНИЦ\n";
```

```
cout << "\nИСХОДНАЯ МАТРИЦА:\n";
```

```
printMatrix(reducedMatrix);
```

```
int lowerBound = 0;
```

```
for (int step = 1; step <= N; step++)  
{
```

```
    cout << "ШАГ "  
        << step  
        << endl;
```

```
    int rowReduction = reduceRows(reducedMatrix);
```

```
    cout << "\nРЕДУКЦИЯ СТРОК = "  
        << rowReduction  
        << endl;
```

```
    int colReduction = reduceCols(reducedMatrix);
```

```
    cout << "РЕДУКЦИЯ СТОЛБЦОВ = "  
        << colReduction  
        << endl;
```

```
    lowerBound += rowReduction + colReduction;
```

```
    cout << "\nТЕКУЩАЯ НИЖНЯЯ ГРАНИЦА = "  
        << lowerBound
```

```

        << endl;

    printMatrix(reducedMatrix);

    printPenalties(reducedMatrix);

    Edge chosen = chooseBestEdge(reducedMatrix);

    if (chosen.from == -1)
        break;

    resultPath.push_back(chosen);

    removeRowCol(reducedMatrix,
        chosen.from,
        chosen.to);

    cout << "\nРЕБРО "
        << city[chosen.from]
        << " -> "
        << city[chosen.to]
        << " ДОБАВЛЕНО В МАРШРУТ\n";
}

printTree();

cout << "ФИНАЛЬНЫЙ МАРШРУТ:\n\n";

for (auto e : resultPath)
{
    cout << city[e.from]
        << " -> ";
}

if (!resultPath.empty())
    cout << city[resultPath[0].from];

```

```

int totalCost = calculateFinalCost();

cout << "\n\nОБЩАЯ СТОИМОСТЬ = "
      << totalCost
      << endl;

return 0;
}

```

Результаты работы:

```

ЗАДАЧА КОММИВОЯЖЕРА
МЕТОД ВЕТВЕЙ И ГРАНИЦ

ИСХОДНАЯ МАТРИЦА:

РЕДУЦИРОВАННАЯ МАТРИЦА:

      A      B      C      D      E      F
A  INF  12      7      18      5      9
B  12  INF      4      6      14     11
C   7   4  INF      8     10     15
D  18   6   8  INF      3     13
E   5   5  10      3  INF     16
F   9  11  15     13     16  INF

ШАГ 1

РЕДУКЦИЯ СТРОК = 28
РЕДУКЦИЯ СТОЛБЦОВ = 4

ТЕКУЩАЯ НИЖНЯЯ ГРАНИЦА = 32

РЕДУЦИРОВАННАЯ МАТРИЦА:

      A      B      C      D      E      F
A  INF      7      2     13      0      0
B   8  INF      0      2     10      3
C   3   0  INF      4      6      7
D  15   3   5  INF      0      6
E   2  11   7   0  INF      9
F   0   2   6   4   7  INF

ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК:
(A -> E) = 0
(A -> F) = 3
(B -> C) = 4
(C -> B) = 5
(D -> E) = 3
(E -> D) = 4
(F -> A) = 4

ВЫБРАНО РЕБРО: C -> B
ШТРАФ = 5

РЕБРО C -> B ДОБАВЛЕНО В МАРШРУТ
ШАГ 2

РЕДУКЦИЯ СТРОК = 2
РЕДУКЦИЯ СТОЛБЦОВ = 2

ТЕКУЩАЯ НИЖНЯЯ ГРАНИЦА = 36

РЕДУЦИРОВАННАЯ МАТРИЦА:

      A      B      C      D      E      F
A  INF  INF      0     13      0      0
B   6  INF  INF      0      8      1
C  INF  INF  INF  INF  INF  INF  INF
D  15  INF      3  INF      0      6
E   2  INF      5      0  INF      9
F   0  INF      4      4      7  INF

ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК:
(A -> C) = 3
(A -> E) = 0
(A -> F) = 1
(B -> D) = 1
(D -> E) = 3
(E -> D) = 2
(F -> A) = 6

ВЫБРАНО РЕБРО: F -> A
ШТРАФ = 6

РЕБРО F -> A ДОБАВЛЕНО В МАРШРУТ
ШАГ 3

РЕДУКЦИЯ СТРОК = 0
РЕДУКЦИЯ СТОЛБЦОВ = 1

ТЕКУЩАЯ НИЖНЯЯ ГРАНИЦА = 37

РЕДУЦИРОВАННАЯ МАТРИЦА:

      A      B      C      D      E      F
A  INF  INF  INF      0     13      0  INF
B  INF  INF  INF  INF      0      8      0
C  INF  INF  INF  INF  INF  INF  INF
D  INF  INF      3  INF      0      5
E  INF  INF      5      0  INF      8
F  INF  INF  INF  INF  INF  INF  INF

ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК:
(A -> C) = 3
(A -> E) = 0
(B -> D) = 0
(B -> F) = 5
(D -> E) = 3
(E -> D) = 5

ВЫБРАНО РЕБРО: B -> F
ШТРАФ = 5

РЕБРО B -> F ДОБАВЛЕНО В МАРШРУТ
ШАГ 4

РЕДУКЦИЯ СТРОК = 0

```

```
РЕДУКЦИЯ СТРОК = 0
РЕДУКЦИЯ СТОЛБЦОВ = 0

ТЕКУЩАЯ НИЖНЯЯ ГРАНИЦА = 37
```

РЕДУЦИРОВАННАЯ МАТРИЦА:

	A	B	C	D	E	F
A	INF	INF	0	13	0	INF
B	INF	INF	INF	INF	INF	INF
C	INF	INF	INF	INF	INF	INF
D	INF	INF	3	INF	0	INF
E	INF	INF	5	0	INF	INF
F	INF	INF	INF	INF	INF	INF

ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК:

```
(A -> C) = 3
(A -> E) = 0
(D -> E) = 3
(E -> D) = 18
```

ВЫБРАНО РЕБРО: E -> D

ШТРАФ = 18

РЕБРО E -> D ДОБАВЛЕНО В МАРШРУТ
ШАГ 5

```
РЕДУКЦИЯ СТРОК = 3
РЕДУКЦИЯ СТОЛБЦОВ = 0
```

ТЕКУЩАЯ НИЖНЯЯ ГРАНИЦА = 40

РЕДУЦИРОВАННАЯ МАТРИЦА:

	A	B	C	D	E	F
A	INF	INF	0	INF	0	INF
B	INF	INF	INF	INF	INF	INF
C	INF	INF	INF	INF	INF	INF
D	INF	INF	0	INF	INF	INF
E	INF	INF	INF	INF	INF	INF
F	INF	INF	INF	INF	INF	INF

ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК:

```
(A -> C) = 0
(A -> E) = 0
(D -> C) = 0
```

ВЫБРАНО РЕБРО: A -> C

ШТРАФ = 0

РЕБРО A -> C ДОБАВЛЕНО В МАРШРУТ
ШАГ 6

```
РЕДУКЦИЯ СТРОК = 0
РЕДУКЦИЯ СТОЛБЦОВ = 0
```

ТЕКУЩАЯ НИЖНЯЯ ГРАНИЦА = 40

РЕДУЦИРОВАННАЯ МАТРИЦА:

	A	B	C	D	E	F
A	INF	INF	INF	INF	INF	INF
B	INF	INF	INF	INF	INF	INF
C	INF	INF	INF	INF	INF	INF
D	INF	INF	INF	INF	INF	INF
E	INF	INF	INF	INF	INF	INF
F	INF	INF	INF	INF	INF	INF

ШТРАФЫ ДЛЯ НУЛЕВЫХ КЛЕТОК:

Нулевые элементы больше не найдены.
ФИНАЛЬНОЕ ДЕРЕВО МАРШРУТА

C -> B

|

v

F -> A

|

v

B -> F

|

v

E -> D

|

v

A -> C

ФИНАЛЬНЫЙ МАРШРУТ:

C -> F -> B -> E -> A -> C

ОБЩАЯ СТОИМОСТЬ = 34

C:\Users\Elena Lapteva\source\repos\commivoyajore\x64\De
Чтобы автоматически закрывать консоль при остановке отла
".
Нажмите любую клавишу, чтобы закрыть это окно:

UML

Задача коммивояжера
Метод ветвей и границ

